



Projeto e Análise de Algoritmos

Estruturas de dados em C++, Java e Python

André Tavares da Silva
andre.silva@udesc.br

Estruturas em C++

- A STL (Standard Template Library) em C++ é uma biblioteca poderosa que fornece estruturas de dados genéricas (contêineres), algoritmos e iteradores, otimizados para alto desempenho e reutilização de código. Apesar de não fazer parte da linguagem (sintaxe), ela pode ser considerada parte integrante do C++, já que é incorporada à biblioteca padrão (std).
- Os contêineres (containers) fornecem estruturas de dados para armazenar objetos, como: `vector` (array dinâmico), `list` (lista duplamente encadeada), `map` (array associativo), `set`, `stack` (pilha) e `queue` (fila).
- Os algoritmos possuem funções padronizadas para manipular contêineres, como `sort()` (ordenar), `find()` (buscar), `reverse()` e `binary_search()`.

Exemplo em C++

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```
int main() {
    std::vector<int> v = {5, 2, 8, 1};
    v.push_back(10); // Adiciona elemento
    std::sort(v.begin(), v.end()); // Ordena
    for(int x : v) std::cout << x << " "; // Imprime: 1 2 5 8 10
    return 0;
}
```

Estruturas em Java e Python

- As estruturas equivalentes em Python e Java seguem lógicas muito parecidas com a STL, mas com nomes e comportamentos específicos:
- Array linear dinâmico (`std::vector`):
 - Python: `list`. É a estrutura padrão para sequências mutáveis.
 - Java: `ArrayList`. Implementa a interface `List` e auto-redimensiona
- Lista duplamente encadeada (`std::list`)
 - Python: `collections.deque`. para inserções e remoções rápidas.
 - Java: `LinkedList`. uma implementação clássica de lista ligada
- Conjunto / elementos únicos (`std::set` e `std::unordered_set`)
 - Python: `set` garante elementos únicos usando *hashing* para buscas rápidas. Para estruturas como `std::set` do C++ e `TreeSet` do Java é preciso usar bibliotecas externas como `sortedcontainers`.
 - Java: `HashSet` (equivalente ao `unordered_set`) ou `TreeSet` (equivalente ao `std::set` ordenado).

Estruturas em C++, Java e Python

- Em relação às estruturas de dados estudadas na disciplina:

Linguagem	Vetor/Array linear	Lista duplamente encadeada	Árvore Rubro-Negra	Hash
C++	std::vector	std::list	std::set	std::unordered_set std::unordered_map
Java	List e ArrayList	LinkedList	TreeSet	HashSet HashMap
Python	list	collections.deque	sortedcontainers	set dict

Complexidade de tempo e espaço

Linguagem	Vetor/Array linear	Lista duplamente encadeada	Árvore Rubro-Negra	Hash
Acesso aleatório	$O(1)$	$O(n)$	$O(\log n)$	$O(1)^*$
Busca (não ordenado)	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$
inserção/ remoção no início	$O(n)$	$O(1)$	$O(\log n)$	$O(1)$
inserção/ remoção no fim	$O(1)$	$O(1)$	$O(\log n)$	$O(1)$
Espaço	baixo	alto	alto	médio-alto

* tem mais um custo pequeno da conversão da chave

E agora... qual eu uso?

- Em geral, uma lista baseada em **vetor dinâmico** costuma ser a opção padrão na maioria dos casos. Ela oferece acesso rápido por índice, boa performance para leitura e excelente eficiência quando os elementos são adicionados principalmente no final. Por isso, é amplamente usada em situações como catálogos, feeds, históricos e qualquer cenário em que se percorre os dados com frequência. Sua principal desvantagem aparece quando há muitas inserções ou remoções no início ou no meio da estrutura, pois isso exige deslocamento de elementos e pode se tornar custoso.

Muito usado no dia-a-dia

E agora... qual eu uso?

- Estruturas baseadas em **listas encadeadas** são mais adequadas quando há muitas inserções e remoções nas extremidades ou em posições já conhecidas. Elas são úteis em cenários como filas, *deques* ou sistemas de processamento sequencial. No entanto, possuem desempenho ruim para acesso aleatório e busca, já que exigem percorrer os elementos um a um. Por isso, são menos usadas na prática em comparação com estruturas baseadas em *arrays*.

Mais eficiente para reorganizar os elementos na lista

E agora... qual eu uso?

- Para buscas rápidas e verificação de existência, estruturas baseadas em tabela **hash** são geralmente as melhores escolhas. Elas permitem inserções, remoções e consultas em tempo médio constante, sendo ideais para casos como remoção de duplicatas, cache de valores, controle de itens já processados ou verificação de pertencimento. Em contrapartida, não mantêm os elementos ordenados.

Mais rápido para busca

E agora... qual eu uso?

- Já quando é necessário manter os elementos sempre ordenados ou realizar consultas baseadas em ordem, como encontrar o menor, maior, próximo ou intervalos de valores, entram as estruturas baseadas em **árvores balanceadas**. Elas garantem operações em tempo logarítmico e mantêm os dados automaticamente ordenados, sendo muito usadas em agendas, rankings e sistemas que dependem de ordenação dinâmica. O custo é maior do que o das tabelas hash, mas em troca há suporte nativo à ordenação.



Projeto e Análise de Algoritmos

Estruturas de dados em C++, Java e Python

André Tavares da Silva
andre.silva@udesc.br